7

Evaluating LLMs

LLM evaluation is a crucial process used to assess the performance and capabilities of LLM models. It can take multiple forms, such as multiple-choice question answering, open-ended instructions, and feedback from real users. Currently, there is no unified approach to measuring a model's performance but there are patterns and recipes that we can adapt to specific use cases.

While general-purpose evaluations are the most popular ones, with benchmarks like **Massive Multi-Task Language Understanding** (**MMLU**) or LMSYS Chatbot Arena, domain- and task-specific models benefit from more narrow approaches. This is particularly true when dealing with entire LLM systems (as opposed to models), often centered around a **retrieval-augmented generation** (**RAG**) pipeline. In these scenarios, we need to expand our evaluation framework to encompass the entire system, including new modules like retrievers and post-processors.

In this chapter, we will cover the following topics:

- Model evaluation
- RAG evaluation
- Evaluating TwinLlama-3.1-8B

By the end of this chapter, you will know the most popular LLM evaluations and how to evaluate models and RAG systems using different

techniques.

Model evaluation

In model evaluation, the objective is to assess the capabilities of a single model without any prompt engineering, RAG pipeline, and so on.

This evaluation is essential for several reasons, such as selecting the most relevant LLM or making sure that the fine-tuning process actually improved the model. In this section, we will compare ML and LLM evaluation to understand the main differences between these two fields. We will then explore benchmarks for general-purpose, domain-specific, and task-specific models.

Comparing ML and LLM evaluation

ML evaluation is centered on assessing the performance of models designed for tasks like prediction, classification, and regression. Unlike the evaluation of LLMs, which often focuses on how well a model understands and generates language, ML evaluation is more concerned with how accurately and efficiently a model can process structured data to produce specific outcomes.

This difference comes from the nature of the tasks these models handle. ML models are generally designed for narrowly defined problems, such as predicting stock prices or detecting outliers, which often involve numerical or categorical data, making the evaluation process more straightforward. On the other hand, LLMs are tasked with interpreting and generating language, which adds a layer of subjectivity to the evaluation process. Instead of relying solely on numerical benchmarks, LLM evaluation requires a more nuanced approach and often incorporates qualitative assessments, examining how well the model produces coherent, relevant, and contextually accurate responses in natural language.

In particular, we can see three key differences in how these models work, which impact the evaluation process:

- **Numerical metrics**: Evaluating ML models typically involves measuring objective performance metrics, such as accuracy, precision, recall, or mean squared error, depending on the type of task at hand. This is less clear with LLMs, which can handle multiple tasks (hence,

multiple evaluations) and can rarely rely on the same numerical metrics.

- **Feature engineering**: In traditional ML, a critical part of the process involves manually selecting and transforming relevant data features before training the model. Evaluating the success of this feature engineering often becomes part of the broader model evaluation. LLMs, however, are designed to handle raw text data directly, reducing the need for manual feature engineering.
- **Interpretability**: With ML models, it is easier to interpret why a model made certain predictions or classifications, and this interpretability can be a core part of their evaluation. This direct interpretation is not possible with LLMs. However, requesting explanations during the generation process can give insights into the model's decision-making process.

In the following section, we will see a more fine-grained exploration of different types of LLMs. While evaluating general-purpose models is fairly disconnected from ML evaluation, task-specific LLMs are more closely aligned with traditional ML.

General-purpose LLM evaluations

General-purpose evaluations refer to metrics dedicated to base and general-purpose fine-tuned models. They cover a breadth of capabilities that are correlated with knowledge and usefulness without focusing on specific tasks or domains. This allows developers to get an overview of these capabilities, compare themselves with competitors, and identify strengths and weaknesses. Based on these results, it is possible to tweak the dataset and hyperparameters, or even modify the architecture.

We can broadly categorize general-purpose evaluations in three phases: during pre-training, after pre-training, and after fine-tuning.

During pre-training, we closely monitor how the model learns, as shown at the end of *Chapter 5*. The most straightforward metrics are low-level and correspond to how models are trained:

- **Training loss**: Based on the cross-entropy loss, measures the difference between the model's predicted probability distribution and the true distribution of the next token
- **Validation loss**: Calculates the same loss as training loss, but on a held-out validation set to assess generalization
- **Perplexity**: Exponential of the cross-entropy loss, representing how "surprised" the model is by the data (lower is better)
- **Gradient norm**: Monitors the magnitude of gradients during training to detect potential instabilities or vanishing/exploding gradients

It's also possible to include benchmarks like HellaSwag (common sense reasoning) during this stage but there's a risk of overfitting these

evaluations.

After pre-training, it is common to use a suite of evaluations to evaluate the base model. This suite can include internal and public benchmarks. Here's a non-exhaustive list of common public pre-training evaluations:

- **MMLU (knowledge)**: Tests models on multiple-choice questions across 57 subjects, from elementary to professional levels
- **HellaSwag (reasoning)**: Challenges models to complete a given situation with the most plausible ending from multiple choices
- **ARC-C (reasoning)**: Evaluates models on grade-school-level multiple-choice science questions requiring causal reasoning
- **Winogrande (reasoning)**: Assesses common sense reasoning through pronoun resolution in carefully crafted sentences
- **PIQA (reasoning)**: Measures physical common sense understanding through questions about everyday physical interactions

Many of these datasets are also used to evaluate general-purpose fine-tuned models. In this case, we focus on the difference in a given score between the base and the fine-tuned model. For example, bad fine-tuning can degrade the knowledge of the model, measured by MMLU. On the contrary, a good one might instill even more knowledge and increase the MMLU score.

This can also help identify any contamination issues, where the model might have been fine-tuned on data that is too close to a test set. For instance, improving the MMLU score of a base model by 10 points during the fine-tuning phase is unlikely. This is a sign that the instruction data might be contaminated.

In addition to these pre-trained evaluations, fine-tuned models also have their own benchmarks. Here, we use the term “fine-tuned model” to designate a model that has been trained with **supervised fine-tuning** (**SFT**) and preference alignment. These benchmarks target capabilities connected to the ability of fine-tuned models to understand and answer questions. In particular, they test instruction-following, multi-turn conversation, and agentic skills:

- **IFEval (instruction following)**: Assesses a model’s ability to follow instructions with particular constraints, like not outputting any commas in your answer
- **Chatbot Arena (conversation)**: A framework where humans vote for the best answer to an instruction, comparing two models in head-to-head conversations
- **AlpacaEval (instruction following)**: Automatic evaluation for fine-tuned models that is highly correlated with Chatbot Arena
- **MT-Bench (conversation)**: Evaluates models on multi-turn conversations, testing their ability to maintain context and provide coherent responses
- **GAIA (agentic)**: Tests a wide range of abilities like tool use and web browsing, in a multi-step fashion

Understanding how these evaluations are designed and used is important to choose the best LLM for your application. For example, if you want to fine-tune a model, you want the best base model in terms of knowledge and reasoning for a given size. This allows you to compare the capabilities of different LLMs and pick the one that will offer the strongest foundation for your fine-tuning.

Even if you don’t want to fine-tune a model, benchmarks like Chatbot Arena or IFEval are a good way to compare different instruct models. For instance, you want great conversational abilities if you’re building a chatbot. However, this is not necessary if your end goal is something like information extraction from unstructured documents. In this case, you will benefit more from excellent instruction-following skills to understand and execute tasks.

While these benchmarks are popular and useful, they also suffer from inherent flaws. For example, public benchmarks can be gamed by training models on test data or samples that are very similar to benchmark datasets. Even human evaluation is not perfect and is often biased toward long and confident answers, especially when they’re nicely formatted (e.g., using Markdown). On the other hand, private test sets have not been scrutinized as much as public ones and might have their own issues and biases.

This means that benchmarks are not a single source of truth but should be used as signals. Once multiple evaluations provide a similar answer, you can raise your confidence level about the real capabilities of a model.

Domain-specific LLM evaluations

Domain-specific LLMs don’t have the same scope as general-purpose models. This is helpful to target more fine-grained capabilities with more depth than the previous benchmarks.

Within the category, the choice of benchmarks entirely depends on the domain in question. For common applications like a language-specific model or a code model, it is recommended to search for relevant evaluations and even benchmark suites. These suites encompass different benchmarks and are designed to be reproducible. By targeting different aspects of a domain, they often capture domain performance more accurately.

To illustrate this, here is a list of domain-specific evaluations with leaderboards on the Hugging Face Hub:

- **Open Medical-LLM Leaderboard**: Evaluates the performance of LLMs in medical question-answering tasks. It regroups 9 metrics, with 1,273 questions from the US medical license exams (MedQA), 500 questions from PubMed articles (PubMedQA), 4,183 questions from Indian medical entrance exams (MedMCQA), and 1,089 questions from 6 sub-categories of MMLU (clinical knowledge, medical genetics, anatomy, professional medicine, college biology, and college medicine).
- **BigCodeBench Leaderboard**: Evaluates the performance of code LLMs, featuring two main categories: BigCodeBench-Complete for code completion based on structured docstrings, and BigCodeBench-

Instruct for code generation from natural language instructions. Models are ranked by their Pass@1 scores using greedy decoding, with an additional Elo rating for the Complete variant. It covers a wide range of programming scenarios that test LLMs' compositional reasoning and instruction-following capabilities.

- **Hallucinations Leaderboard**: Evaluates LLMs' tendency to produce false or unsupported information across 16 diverse tasks spanning 5 categories. These include *Question Answering* (with datasets like NQ Open, TruthfulQA, and SQuADv2), *Reading Comprehension* (using TriviaQA and RACE), *Summarization* (employing HaluEval Summ, XSum, and CNN/DM), *Dialogue* (featuring HaluEval Dial and FaithDial), and *Fact Checking* (utilizing MemoTrap, SelfCheckGPT, FEVER, and TrueFalse). The leaderboard also assesses instruction-following ability using IFEval.
- **Enterprise Scenarios Leaderboard**: Evaluates the performance of LLMs on six real-world enterprise use cases, covering diverse tasks relevant to business applications. The benchmarks include FinanceBench (100 financial questions with retrieved context), Legal Confidentiality (100 prompts from LegalBench for legal reasoning), Writing Prompts (creative writing evaluation), Customer Support Dialogue (relevance in customer service interactions), Toxic Prompts (safety assessment for harmful content generation), and Enterprise PII (business safety for sensitive information protection). Some test sets are closed-source to prevent gaming of the leaderboard. The evaluation focuses on specific capabilities such as answer accuracy, legal reasoning, creative writing, contextual relevance, and safety measures, providing a comprehensive assessment of LLMs' suitability for enterprise environments.

Leaderboards can have different approaches based on their domain. For example, BigCodeBench is significantly different from others because it relies on only two metrics that sufficiently capture the entire domain. On the other hand, the Hallucinations Leaderboard regroups 16 metrics, including many general-purpose evaluations. It shows that in addition to custom benchmarks, reusing general-purpose ones can complete your own suite.

In particular, language-specific LLMs often reuse translated versions of general-purpose benchmarks. This can be completed with original evaluations in the native language. While some of these benchmarks use machine translation, it is better to rely on human-translated evaluations to improve their quality. We selected the following three task-specific leaderboards and their respective evaluation suites to give you an idea of how to build your own:

- **OpenKo-LLM Leaderboard**: Evaluates the performance of Korean LLMs using nine metrics. These metrics are a combination of general-purpose benchmarks translated into Korean (GPQA, Winogrande, GSM8K, EQ-Bench, and IFEval) and custom evaluations (Knowledge, Social Value, Harmlessness, and Helpfulness).
- **Open Portuguese LLM Leaderboard**: Evaluates the performance of Portuguese language LLMs using nine diverse benchmarks. These benchmarks include educational assessments (ENEM with 1,430 questions, and BLUEX with 724 questions from university entrance exams), professional exams (OAB Exams with over 2,000 questions), language understanding tasks (ASSIN2 RTE and STS, FAQUAD NLI), and social media content analysis (HateBR with 7,000 Instagram comments, PT Hate Speech with 5,668 tweets, and tweetSentBR).

- **Open Arabic LLM Leaderboard**: Evaluates the performance of Arabic language LLMs using a comprehensive set of benchmarks, including both native Arabic tasks and translated datasets. The leaderboard features two native Arabic benchmarks: AlGhafa and Arabic-Culture-Value-Alignment. Additionally, it incorporates 12 translated benchmarks covering various domains, such as MMLU, ARC-Challenge, HellaSwag, and PIQA.

Both general-purpose and domain-specific evaluations are designed with three main principles. First, they should be complex and challenge models to distinguish good and bad outputs. Second, they should be diverse and cover as many topics and scenarios as possible. When one benchmark is not enough, additional ones can create a stronger suite. Finally, they should be practical and easy to run. This is more connected to evaluation libraries, which can be more or less complex to work with. We recommend lm-evaluation-harness (github.com/EleutherAI/lm-evaluation-harness) from Eleuther AI and lighteval (github.com/huggingface/lighteval) from Hugging Face to run your benchmarks.

Task-specific LLM evaluations

While general-purpose and domain-specific evaluations indicate strong base or instruct models, they cannot provide insights into how well these models work for a given task. This requires benchmarks specifically designed for this purpose, measuring downstream performance.

Because of their narrow focus, task-specific LLMs can rarely rely on pre-existing evaluation datasets. This can be advantageous because their outputs also tend to be more structured and easier to evaluate using traditional ML metrics. For example, a summarization task can leverage the **Recall-Oriented Understudy for Gisting Evaluation** (**ROUGE**) metric, which measures the overlap between the generated text and reference text using n-grams.

Likewise, classification tasks also benefit from it and use the following classic metrics, among others:

- **Accuracy**: Accuracy refers to the proportion of correctly predicted instances compared to the total instances. It's particularly useful for tasks with categorical outputs or where there is a clear distinction between right and wrong answers, such as **named entity recognition** (**NER**).
- **Precision**: The ratio of true positive predictions to the total positive predictions made by the model.
- **Recall**: The ratio of true positive predictions to the total actual positive instances.
- **F1 Score**: The harmonic mean of precision and recall, used to balance both metrics. These are particularly useful in tasks such as

classification or entity extraction.

When the task cannot be directly mapped to a traditional ML task, it is possible to create a custom benchmark. This benchmark can be inspired by general-purpose and domain-specific evaluation datasets. A common and successful pattern is the use of multiple-choice question answering. In this framework, the instruction consists of a question with several options. See the following example with a question from the MMLU dataset (abstract algebra):

Instruction Find the degree for the given field extension Q(sqrt(2), sqrt(3)) over Q. A. 0 B. 4 C. 2 D. 6
Output B

Table 7.1: Example from the MMLU dataset

There are two main ways of evaluating models with this scheme—text generation and log-likelihood evaluations:

- The first approach involves having the model generate text responses and comparing those to predefined answer choices. For example, the model generates a letter (A, B, C, or D) as its answer, which is then

checked against the correct answer. This method tests the model's ability to produce coherent and accurate responses in a format similar to how it would be used in real-world applications.

- Evaluation using probabilities, on the other hand, looks at the model's predicted probabilities for different answer options without requiring text generation. For MMLU, lm-evaluation-harness compares the probabilities for the full text of each answer choice. This approach allows for a more nuanced assessment of the model's understanding, as it can capture the relative confidence the model has in different options, even if it wouldn't necessarily generate the exact correct answer text.

For simplicity, we recommend the text-generation version of the evaluation that mimics human test-taking. It is easier to implement, and generally more discriminative, as low-quality models tend to overperform on probability-based evaluations. You can adapt this technique to quiz your models about a particular task, and even expand it to specific domains.

Conversely, if the task is too open-ended, traditional ML metrics and multiple-choice question answering might not be relevant. In this scenario, the LLM-as-a-judge technique introduced in *Chapter 5* can be used to evaluate the quality of the answers. If you have ground-truth answers, providing them as additional context improves the accuracy of the evaluation. Otherwise, defining different dimensions (such as relevance or toxicity, depending on your task) can also ground the evaluation in more interpretable categories.

It is recommended to use large models for evaluation and to iteratively refine your prompt. In this process, the explanations outputted by the model

are important for understanding errors in its reasoning and fixing them through additional prompt engineering.

In order to easily parse answers, one can specify a structure in the instruction or use some kind of structured generation (like Outlines or OpenAI's JSON mode). Here is an example of an instruction with a structure:

```
You are an evaluator who assesses the quality of an answer to an instruction.

Your goal is to provide a score that represents how well the answer addresses the instruction.

You will use a scale of 1 to 4, where each number represents the following:

1. The answer is not relevant to the instruction.

2. The answer is relevant but not helpful.

3. The answer is relevant and helpful but could be more detailed.

4. The answer is relevant, helpful, and detailed.

Please provide your evaluation as follows:

##Evaluation##

Explanation: (analyze the relevant, helpfulness, and complexity of the answer)
```

`Total rating: (final score as a number between 1 and 4)` **Instruction**: {instruction} **Answer**: {answer} ##Evaluation## Explanation:

Table 7.2: Example of general-purpose LLM-as-a-judge prompt for answer evaluation

Naturally, you can tweak the scale, add a ground-truth answer to this prompt, and customize it for your own use cases.

However, judge LLMs can exhibit biases favoring assertive or verbose responses, potentially overrating answers that sound more confident but are less accurate. They may also lack domain expertise for specialized topics, leading to misjudgments. Consistency is also a concern, as LLMs might score similar responses differently. Additionally, they could have implicit preferences for certain writing styles unrelated to actual answer quality. To mitigate these issues, it's possible to combine LLM evaluations with other metrics, use multiple judges, and carefully design prompts to address biases.

Once a model has been properly evaluated and works as intended, it might be included within a broader system. In the next section, we will see how systems change the evaluation framework.

RAG evaluation

While traditional LLM evaluation focuses on the model's inherent capabilities, RAG evaluation requires a more comprehensive approach that considers both the model's generative abilities and its interaction with external information sources.

RAG systems combine the strengths of LLMs with information retrieval mechanisms, allowing them to generate responses that are not only coherent and contextually appropriate but also grounded in up-to-date, externally sourced information. This makes RAG particularly valuable in fields where current and accurate information is crucial, such as news reporting, research, and customer support.

The evaluation of RAG systems goes beyond assessing a standalone LLM. It requires examining the entire system's performance, including:

- **Retrieval accuracy**: How well does the system fetch relevant information?
- **Integration quality**: How effectively is the retrieved information incorporated into the generated response?
- **Factuality and relevance**: Does the final output address the query appropriately while seamlessly blending retrieved and generated content?

Key metrics for RAG evaluation include retrieval precision and recall, which measure the accuracy and comprehensiveness of the retrieved information. Additionally, the quality of integration between retrieved data and generated text is crucial, as is the overall factuality and coherence of the output.

To illustrate how these metrics are applied in practice, consider a RAG system designed for a customer support chatbot in an e-commerce setting. In this scenario, the user asks “What’s your return policy for laptops purchased during the holiday sale?” The RAG pipeline finds relevant documents on the electronics return policy and documents on holiday sale terms. This additional context is appended at the end of the question, and the model uses it to respond:

`For laptops purchased during our holiday sale, you have an extended return period of 60 days from the date of purchase. This is longer than our standard 30-day return policy for electronics. Please ensure the laptop is in its original packaging with all accessories to be eligible for a full refund.`

Table 7.3: Example of output from a RAG pipeline designed for customer support

In this pipeline, we can evaluate if the retrieved documents correspond to what was expected (retrieval accuracy). We can also measure the difference between responses with and without additional context (integration quality). Finally, we can assess whether the output is relevant and grounded in the information provided by the documents (factuality and relevance).

In this section, we will cover two methods to evaluate how well RAG models incorporate external information into their responses.

Ragas

Retrieval-Augmented Generation Assessment (**Ragas**) is an open-source toolkit designed to provide developers with a comprehensive set of tools for RAG evaluation and optimization. It's designed around the idea of **metrics-driven development** (**MDD**), a product development approach that relies on data to make well-informed decisions, involving the ongoing monitoring of essential metrics over time to gain valuable insights into an application's performance. By embracing this methodology, Ragas enables developers to objectively assess their RAG systems, identify areas for improvement, and track the impact of changes over time.

One of the key capabilities of Ragas is its ability to synthetically generate diverse and complex test datasets. This feature addresses a significant pain point in RAG development, as manually creating hundreds of questions, answers, and contexts is both time-consuming and labor-intensive. Instead, it uses an evolutionary approach paradigm inspired by works like Evol-Instruct to craft questions with varying characteristics such as reasoning complexity, conditional elements, and multi-context requirements. This approach ensures a comprehensive evaluation of different components within the RAG pipeline.

Additionally, Ragas can generate conversational samples that simulate chat-based question-and-follow-up interactions, allowing developers to evaluate their systems in more realistic scenarios.

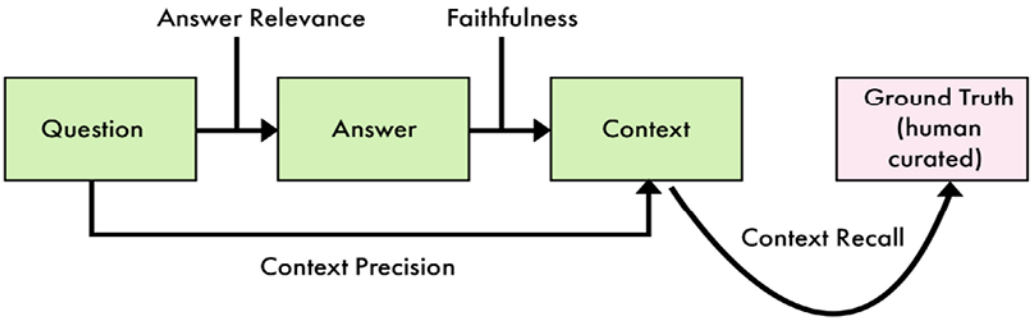


Figure 7.1: Overview of the Ragas evaluation framework

As illustrated in *Figure 7.1*, Ragas provides a suite of LLM-assisted evaluation metrics designed to objectively measure different aspects of RAG system performance. These metrics include:

- **Faithfulness**: This metric measures the factual consistency of the generated answer against the given context. It works by breaking down the answer into individual claims and verifying if each claim can be inferred from the provided context. The faithfulness score is calculated as the ratio of verifiable claims to the total number of claims in the answer.
- **Answer relevancy**: This metric evaluates how pertinent the generated answer is to the given prompt. It uses an innovative approach where an LLM is prompted to generate multiple questions based on the answer and then calculates the mean cosine similarity between these generated questions and the original question. This method helps identify answers that may be factually correct but off-topic or incomplete.
- **Context precision**: This metric evaluates whether all the ground-truth relevant items present in the contexts are ranked appropriately. It considers the position of relevant information within the retrieved context, rewarding systems that place the most pertinent information at the top.

- **Context recall**: This metric measures the extent to which the retrieved context aligns with the annotated answer (ground truth). It analyzes each claim in the ground truth answer to determine whether it can be attributed to the retrieved context, providing insights into the completeness of the retrieved information.

Finally, Ragas also provides building blocks for monitoring RAG quality in production environments. This facilitates continuous improvement of RAG systems. By leveraging the evaluation results from test datasets and insights gathered from production monitoring, developers can iteratively enhance their applications. This might involve fine-tuning retrieval algorithms, adjusting prompt engineering strategies, or optimizing the balance between retrieved context and LLM generation.

Ragas can be complemented with another approach, based on custom classifiers.

ARES

ARES (an automated evaluation framework for RAG systems) is a comprehensive tool designed to evaluate RAG systems. It offers an automated process that combines synthetic data generation with fine-tuned classifiers to assess various aspects of RAG performance, including context relevance, answer faithfulness, and answer relevance.

The ARES framework operates in three main stages: synthetic data generation, classifier training, and RAG evaluation. Each stage is configurable, allowing users to tailor the evaluation process to their specific needs and datasets.

In the synthetic data generation stage, ARES creates datasets that closely mimic real-world scenarios for robust RAG testing. Users can configure this process by specifying document file paths, few-shot prompt files, and output locations for the synthetic queries. The framework supports various pre-trained language models for this task, with the default being google/flan-t5-xxl. Users can control the number of documents sampled and other parameters to balance between comprehensive coverage and computational efficiency.

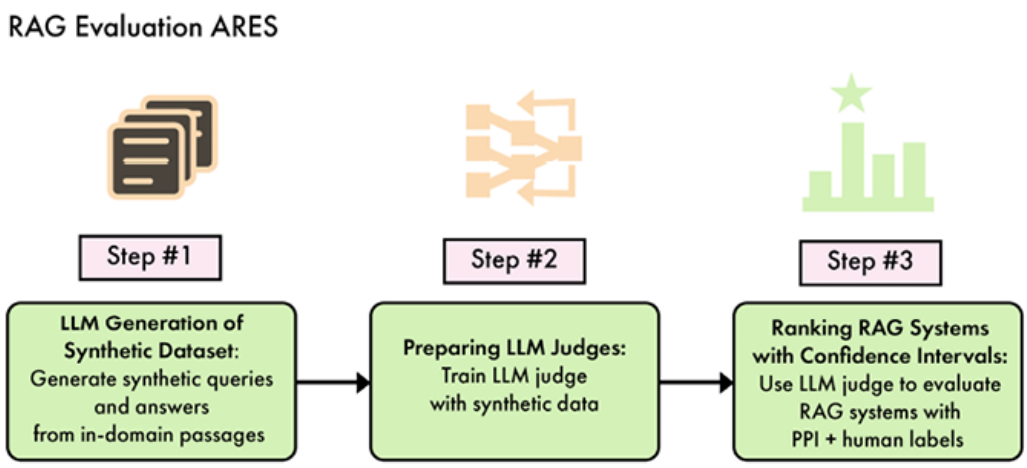


Figure 7.2: Overview of the ARES evaluation framework

The classifier training stage involves creating high-precision classifiers to determine the relevance and faithfulness of RAG outputs. Users can specify the classification dataset (typically generated from the previous stage), test set for evaluation, label columns, and model choice. ARES uses microsoft/deberta-v3-large as the default model but supports other Hugging Face models. Training parameters such as the number of epochs, patience value for early stopping, and learning rate can be fine-tuned to optimize classifier performance.

The final stage, RAG evaluation, leverages the trained classifiers and synthetic data to assess the RAG model's performance. Users provide evaluation datasets, few-shot examples for guiding the evaluation, classifier checkpoints, and gold label paths. ARES supports various evaluation metrics and can generate confidence intervals for its assessments.

ARES offers flexible model execution options, supporting both cloud-based and local runs through vLLM integration. The framework also supports various artifact types (code snippets, documents, HTML, images, and so on), enabling comprehensive evaluation across different RAG system outputs.

In summary, Ragas and ARES complement each other through their distinct approaches to evaluation and dataset generation. Ragas's strength in production monitoring and LLM-assisted metrics can be combined with ARES's highly configurable evaluation process and classifier-based assessments. While Ragas may offer more nuanced evaluations based on LLM capabilities, ARES provides consistent and potentially faster evaluations once its classifiers are trained. Combining them offers a comprehensive evaluation framework, benefiting from quick iterations with Ragas and in-depth, customized evaluations with ARES at key stages.

In the next section, we will create our own evaluation framework to evaluate our task-specific TwinLlama-3.1-8B model.

Evaluating TwinLlama-3.1-8B

In the previous chapters, we created two models fine-tuned to generate high-quality posts and articles: TwinLlama-3.1-8B and TwinLlama-3.1-8B-DPO. Based on this summary, we want to assess their abilities to write text that is both accurate and well-written. In comparison, general-purpose fine-tuned models are accurate thanks to their extensive knowledge but often use overly formal and verbose language. With this fine-tuning, we want to adopt a more natural writing style, based on the original articles from the training set.

Due to the open-ended nature of this problem, we will leverage a judge LLM to evaluate the quality of the generated text. It will take both the instruction and the answer as inputs, and score it on a 1–3 scale based on two criteria:

- **Accuracy**: The degree of factual correctness and comprehensiveness of the information presented in the answer
- **Style**: The appropriateness of the tone and writing style for blog posts or social media content (no formal or academic expressions)

In our evaluation framework, we will use the test split of our instruction dataset to get test instructions. We will feed them to our models and generate answers. These answers will then be evaluated by our judge LLM (GPT-4o-mini), based on a prompt that specifies our criteria. Finally, we will analyze the scores and draw conclusions based on qualitative and quantitative evaluations.

Generating answers

The first step consists of efficiently generating answers for each instruction in our test set. In addition to our two models, we will also use meta-llama/Meta-Llama-3.1-8B-Instruct, the official instruct version of Llama-3.1-8B, as a reference point to better understand the trade-offs we made.

Let's start the first stage of the implementation:

1. We import the relevant libraries, including vLLM for fast generation. This library is a lot faster than transformers for batch generation with local models:

```
from vllm import LLM, SamplingParams
from datasets import load_dataset
from tqdm.auto import tqdm
import gc
```

2. We define a function called `generate_answers` that will process our dataset and generate responses using a specified model. It takes two inputs—the ID of the model we want to use and the name of the test dataset:

```
def generate_answers(model_id, dataset_name):
    dataset = load_dataset(dataset_name, split="test")
```

3. We need to format the raw instructions using the chat template our models have been trained on. Note that Llama-3.1-8B-Instruct has been used with a different template, but it can follow this simple format. Here, we use the same chat template with every model for

simplicity. We map the entire test set to this template with the `format()` function:

```
    def format(sample):
        return "Below is an instruction that describes a tc
    dataset = dataset.map(lambda sample: {"prompt": format(
```

4. Let's initialize the LLM object used by vLLM with a maximum length of 4,096 tokens. We can also specify sampling parameters, which correspond to variables used in the decoding strategy. Here, we use parameters to encourage diversity (high temperature) while removing the most unlikely tokens (`top_p` and `min_p`). Finally, we start the generation by providing the list of prompts with `dataset["prompt"]`:

```
    llm = LLM(model=model_id, max_model_len=4096)
    sampling_params = SamplingParams(temperature=0.8, top_p
    outputs = llm.generate(dataset["prompt"], sampling_para
```

5. This process should take a few minutes with our 334 prompts. Once this is done, we extract the answers from the object that is outputted by vLLM. We then add these answers as a new column to our dataset. This is useful to log the answers and review them later:

```
    answers = [output.outputs[0].text for output in outputs
    dataset = dataset.add_column("answers", answers)
```

6. We save our results to the Hugging Face Hub for easy access later. Then, we clear our GPU memory to prevent running out of space when we process the next model:

```
    print(f"Uploading results for {model_id}")
    dataset.push_to_hub(f"mlabonne/{model_id.split('/')[-1]
    gc.collect()
    return dataset
```

7. We create a list of the three models we want to test. Then, we run our `generate_answers()` function for each of these models, one at a time. This will create and upload a separate set of results for each model:

```
model_ids = [
    'mlabonne/TwinLlama-3.1-8B',
    'mlabonne/TwinLlama-3.1-8B-DPO',
    'meta-llama/Meta-Llama-3.1-8B-Instruct'
]
for model_id in model_ids:
    generate_answers(model_id, "mlabonne/llmtwin")
```

Now that we have the answer generation, we can move on to the evaluation process.

Evaluating answers

To evaluate our answers, we will rely on GPT-4o-mini as a judge. This strategy is similar to what we used for data generation. As a matter of fact, you could adapt it to filter out bad samples during the data generation process. Here, we will score every generated answer from every model in terms of accuracy and style. The average scores will inform us about the quality of our fine-tuning compared to Llama-3.1-8B-Instruct:

1. First, we import the required libraries, including `openai`:

```
import json
from typing import List
from datasets import Dataset, load_dataset
from openai import OpenAI
from tqdm.auto import tqdm
import concurrent.futures
```

2. We then define the `evaluate_answer()` function. This function contains our evaluation prompt, which sets up the context for evaluating answers based on accuracy and style:

```
def evaluate_answer(
    instruction: str, answer: str, client: OpenAI
) -> dict:
    prompt = f"""You are an expert judge. Please evaluate t
1. Accuracy: How factually correct is the information prese
2. Style: Is the tone and writing style appropriate for a b
```

3. In the same prompt, we define our scales for each metric. Those are three-point Likert scales with a precise definition for each score:

```
Accuracy scale:
1 (Poor): Contains factual errors or misleading informatior
2 (Good): Mostly accurate with minor errors or omissions
3 (Excellent): Highly accurate and comprehensive
Style scale:
1 (Poor): Too formal, uses some overly complex words
2 (Good): Good balance of technical content and accessibili
3 (Excellent): Perfectly accessible language for blog/socic
```

4. Finally, we conclude the prompt with two examples to illustrate what we mean by "*complex words*" and "*formal* or *academic language*." We provide the corresponding instruction-answer pair and ask the model to return a response in JSON:

```
Example of bad style: The Llama2 7B model constitutes a not
Example of excellent style: Llama2 7B outperforms the origi
Instruction: {instruction}
Answer: {answer}
Provide your evaluation in JSON format with the following s
{{
    "accuracy": {{
        "analysis": "...",
        "score": 0
    }},
    "style": {{
        "analysis": "...",
        "score": 0
    }}
}}
"""
```

5. This prompt is given as a user query to the GPT-4o-mini model. The system prompt reinforces that we are interested in answer evaluation based on accuracy and style:

```
    completion = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {
                "role": "system",
                "content": "You are a helpful assistant whc
            },
            {"role": "user", "content": prompt},
        ],
        response_format={"type": "json_object"},
        max_tokens=1000,
        temperature=0.8,
    )
```

6. As in the previous chapters, we will batch our requests to speed up the process. This is why we create an `evaluate_batch()` function, which returns a list of parsed structured outputs with their corresponding indices. These indices are important to ensure a correct ordering of the evaluations:

```
def evaluate_batch(batch, start_index):
    client = OpenAI(api_key=OPENAI_KEY)
    return [
        (i, evaluate_answer(instr, ans, client))
        for i, (instr, ans) in enumerate(batch, start=start
    ]
```

7. We can now orchestrate the previous code in the `evaluate_answers()` function. It takes the model ID, number of threads, and batch size as inputs. First, we load the dataset with the generations we previously saved:

```
```

```
def evaluate_answers(model_id: str, num_threads: int = 10,
    dataset = load_dataset(f"mlabonne/{model_id.split('/')[
```

8. We create batches of instruction-answer pairs from our dataset. Each batch contains `batch_size` number of pairs:

```
    batches = [
        (i, list(zip(dataset["instruction"][i:i+batch_size]
        for i in range(0, len(dataset), batch_size)
    ]
```

9. We perform parallel evaluation of batches of instruction-answer pairs using multiple threads. We use parallel processing to evaluate multiple batches simultaneously, speeding up the overall evaluation process. The `ThreadPoolExecutor` submits each batch to `evaluate_batch()`. The results are stored in the evaluations list:

```
    evaluations = [None] * len(dataset)
    with concurrent.futures.ThreadPoolExecutor(max_workers=
        futures = [executor.submit(evaluate_batch, batch, s
        for future in tqdm(concurrent.futures.as_completed(
            for index, evaluation in future.result():
                evaluations[index] = evaluation
```

10. We create a new column with the result of the evaluation process. This column will store the raw JSON output of the judge model, including scores and explanations:

```
    if 'evaluation' in dataset.column_names:
        dataset = dataset.remove_columns(['evaluation'])
    dataset = dataset.add_column("evaluation", evaluations)
```

11. We can directly parse this JSON object with `json.loads()` and try to retrieve the accuracy and style scores that should have been generated. This generation is in best-effort mode, which means that scores are not guaranteed. If there's an error in parsing, we use `None` values as a fallback:

```
accuracy_scores = []
style_scores = []
for evaluation in dataset['evaluation']:
    try:
        eval_dict = json.loads(evaluation) if isinstanc
        accuracy_score = eval_dict['accuracy']['score']
        style_score = eval_dict['style']['score']
        accuracy_scores.append(accuracy_score)
        style_scores.append(style_score)
    except (json.JSONDecodeError, KeyError, TypeError):
        accuracy_scores.append(None)
        style_scores.append(None)
```

12. We add two new columns to store the accuracy and style scores for further analysis:

```
if 'accuracy' in dataset.column_names:
    dataset = dataset.remove_columns(['accuracy'])
dataset = dataset.add_column('accuracy', accuracy_score
if 'style' in dataset.column_names:
    dataset = dataset.remove_columns(['style'])
dataset = dataset.add_column('style', style_scores)
```

13. Let's push the final dataset with generated answers, evaluations, and scores to the Hugging Face Hub:

```
    dataset.push_to_hub(f"mlabonne/{model_id.split('/')[-1]
    return dataset
```

14. We can now call the `evaluate_answers()` function with the three models we selected:

```
model_ids = [
    'mlabonne/TwinLlama-3.1-8B',
    'mlabonne/TwinLlama-3.1-8B-DPO',
    'meta-llama/Meta-Llama-3.1-8B-Instruct'
]
for model_id in model_ids:
    evaluate_answers(model_id)
```

By saving intermediate results, our evaluation framework becomes more robust to failures. It is also easily extendable to other models, datasets, and criteria.

Analyzing results

There are three elements we can review at the end of this evaluation process: model answers, explanations from evaluations, and scores.

While reviewing answers is not scalable, it is a crucial step that can help to identify a lot of common mistakes, such as a wrong chat template or incorrect model. Here, we will focus on a single instruction and read the answers provided by our two models and Llama-3.1-8B-Instruct.

Instruction Discuss the concept of algorithm bias and its implications.
TwinLlama-3.1-8B Algorithm bias refers to the tendency of algorithms to produce outcomes that are skewed or unfair due to underlying assumptions or programming biases. This can have significant implications, particularly in fields like machine learning and AI, where algorithms are relied upon to make critical decisions. If an algorithm is not properly vetted for bias, it may lead to discriminatory outcomes, such as lending practices that disproportionately affect certain demographic groups or recruitment processes that unfairly favor certain candidates. It is essential to ensure that algorithms are designed and implemented with fairness and accuracy in mind to mitigate these risks.
TwinLlama-3.1-8B-DPO

Algorithm bias refers to the tendency of algorithms to produce outcomes that are skewed or unfair due to underlying assumptions or programming biases. This can have significant implications, particularly in fields like machine learning and AI, where algorithms are relied upon to make critical decisions. If an algorithm is not properly vetted for bias, it could lead to discriminatory outcomes, such as denying loans to certain demographic groups or perpetuating stereotypes. Therefore, it is essential to address algorithmic bias proactively to ensure that the technology we develop is equitable and just.

Llama-3.1-8B-Instruct

Algorithm bias, also known as algorithmic bias, refers to the unintended or inherent bias in machine learning models that can affect their performance, accuracy, and fairness. This bias can be caused by various factors, including the data used to train the model, the algorithm itself, and the biases present in the society or culture. One of the most significant implications of algorithm bias is its potential to perpetuate and exacerbate existing social inequalities. For instance, facial recognition software has been shown to be less accurate for people of color, leading to wrongful arrests and discrimination. Similarly, language processing models have been found to be biased towards the language and dialect of the dominant group, making it difficult for marginalized communities to access information and services. Algorithm bias can also have severe consequences in areas such as healthcare, education, and employment. For example, a medical diagnosis algorithm that is biased towards certain demographics may misdiagnose or underdiagnose conditions in other groups, leading to delayed or inadequate treatment. In education, an algorithm that is biased towards certain learning styles may

disproportionately affect students with different learning abilities, leading to unequal opportunities. To mitigate the effects of algorithm bias, it is essential to address the root causes of bias and implement measures to ensure fairness and transparency in machine learning models. This can be achieved through techniques such as data cleansing, regularization, and debiasing, as well as implementing diverse and representative datasets. Additionally, it is crucial to involve diverse stakeholders, including members of marginalized communities, in the development and testing of algorithms to ensure that they are fair and equitable. In conclusion, algorithm bias is a significant concern that can have far-reaching implications for individuals and society as a whole. It is essential to recognize the potential biases in machine learning models and take proactive steps to mitigate their effects. By doing so, we can create more equitable and just systems that benefit everyone, regardless of their background or demographic characteristics.

Table 7.4: Example of generated answers for a given instruction

Immediately, we can see that the answers from our models are very close to each other. This is not the case with Llama-3.1-8B-Instruct's answer, which is extremely verbose and lists many examples. This is typically the kind of answer that is correct but can feel artificial and unpleasant to read because of its length. On the other hand, the DPO model slightly simplifies the language of the SFT version, making it less academic. This is exactly the behavior we want to capture, modifying the writing style but not the actual content of the answer.

Let's now review the evaluations provided by GPT-4o-mini for each answer.

TwinLlama-3.1-8B	
Accuracy The answer accurately defines algorithm bias and highlights its implications in fields like machine learning and AI. It correctly identifies the risks of discriminatory outcomes due to biased algorithms. There are no apparent factual errors, making this a comprehensive response. Score: 3	**Style** The tone is informative and relatively accessible, though it tends towards a formal tone in some sections (e.g., ‘underlying assumptions or programming biases’). While it does use some technical terms appropriately, it could be simplified further for a broader audience, especially for blog or social media content. Score: 2
TwinLlama-3.1-8B-DPO	
Accuracy The answer accurately defines algorithm bias and outlines its implications in critical fields like machine learning and AI. It highlights the potential for discriminatory outcomes, making it a comprehensive overview of the topic without any factual errors.	**Style** The writing style is clear, accessible, and effectively communicates the technical concept of algorithm bias without becoming overly formal. While it uses some technical terms, they are appropriate and explained well for a general audience. Score: 3

Score: 3	
Llama-3.1-8B-Instruct	
Accuracy The answer accurately defines algorithm bias and discusses its causes and implications in various domains such as facial recognition, healthcare, and education. It also correctly identifies methods to mitigate bias, such as involving diverse stakeholders and employing debiasing techniques. Overall, the information is factually correct and comprehensive. Score: 3	**Style** The tone is informative and accessible, with technical terms explained clearly. However, some phrases like ‘unintended or inherent bias’ and ‘implementation of diverse and representative datasets’ could be simplified for broader audience appeal. The writing is generally suitable for blog or social media content, but it could benefit from a more conversational tone. Score: 2

Table 7.5: Evaluations of each answer made by GPT-4o-mini, according to style and accuracy

According to our judge LLM, there is no issue with the accuracy of the answers, which get a perfect score. However, the style is considered too formal for TwinLlama-3.1-8B (SFT) and Llama-3.1-8B-Instruct, with a score of 2. The judge LLM agreed with our previous analysis and assigned a perfect score to TwinLlama-3.1-8B-DPO’s answer for communicating “the technical concept of algorithm bias without becoming overly formal.”

This trend is confirmed by the average scores obtained by each model:

```
TwinLlama-3.1-8B - Accuracy: 2.45
TwinLlama-3.1-8B - Style: 2.04
TwinLlama-3.1-8B-DPO - Accuracy: 2.46
TwinLlama-3.1-8B-DPO - Style: 2.12
Llama-3.1-8B-Instruct - Accuracy: 2.62
Llama-3.1-8B-Instruct - Style: 1.86
```

In terms of accuracy, our two fine-tuned models get similar scores, while Llama-3.1-8B-Instruct achieves the highest accuracy score of 2.62. This suggests that the instruct-tuned Llama model may have a slight edge in providing factually correct information. This is probably due to its extensive post-training process with over 10 million samples (compared to 13,000 in our case).

However, when it comes to style, we see a different pattern. TwinLlama-3.1-8B-DPO leads with a score of 2.12, successfully achieving a more accessible and less formal writing style without sacrificing content quality. TwinLlama-3.1-8B (SFT) follows with 2.04, showing improvement but retaining some formality, while Llama-3.1-8B-Instruct trails with 1.86, tending toward verbosity.

Based on this feedback and the manual review of the generated answers, we can detect mistakes and identify areas for improvement. This is essential for refining the data generation process through additional filtering or augmenting the dataset with missing information. While this first version already shows promising results, iterating over different datasets and models will allow us to significantly outperform our baseline and create the best possible model for our use case.

Summary

In this chapter, we explored LLM evaluation with models and RAG systems. We saw how to interpret classic benchmarks like MMLU to select strong candidates to use or fine-tune. We also detailed how domain-specific and task-specific evaluations work, and how to create our own based on publicly available examples.

We focused on two techniques (multiple-choice question answering and LLM-as-a-judge) as the backbone of these custom evaluation frameworks.

However, models are commonly integrated into broader systems that provide additional context. We introduced two evaluation frameworks for RAG systems, Ragas and ARES. We saw both similarities (for example, synthetic data generation) and differences in how they evaluate RAG systems (context-based metrics versus trained classifiers). Finally, we evaluated TwinLlama-3.1-8B with a judge LLM according to three criteria: relevance, coherence, and conciseness. This provided insights into how we can improve it.

In the next chapter, we will explore inference optimization techniques to improve speed and reduce memory usage, without significantly compromising model performance. We will also delve into optimization methods, model parallelism techniques and examine different quantization approaches.

References

- Lianmin Zheng et al.. “*Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*.” arXiv preprint arXiv:2306.05685, June 2023.
- Aymeric Roucher. “*Using LLM-as-a-judge for an automated and versatile evaluation - Hugging Face Open-Source AI Cookbook*.” huggingface.co, No date found, https://huggingface.co/learn/cookbook/en/llm_judge.
- LangChain. “*Aligning LLM-as-a-Judge with Human Preferences.*” *blog.langchain.dev*, June 26, 2024, https://blog.langchain.dev/aligning-llm-as-a-judge-with-human-preferences/.
- Dan Hendrycks et al.. “*Measuring Massive Multitask Language Understanding*.” arXiv preprint arXiv:2009.03300, September 2020.
- Jeffrey Zhou et al.. “*Instruction-Following Evaluation for Large Language Models*.” arXiv preprint arXiv:2311.07911, November 2023.
- Yann Dubois et al.. “*Length-Controlled AlpacaEval: A Simple Way to Debias Automatic Evaluators*.” arXiv preprint arXiv:2404.04475, April 2024.
- Grégoire Mialon et al.. “*GAIA: a benchmark for General AI Assistants*.” arXiv preprint arXiv:2311.12983, November 2023.
- Giwon Hong et al.. “*The Hallucinations Leaderboard -- An Open Effort to Measure Hallucinations in Large Language Models*.” arXiv preprint arXiv:2404.05904, April 2024.
- Shahul Es et al.. “*RAGAS: Automated Evaluation of Retrieval Augmented Generation*.” arXiv preprint arXiv:2309.15217, September

2023.

- Jon Saad-Falcon et al.. “*ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems*.” arXiv preprint arXiv:2311.09476, November 2023.

Join our book’s Discord space

Join our community’s Discord space for discussions with the authors and other readers:

https://packt.link/llmeng

